Sistemas operativos

Jaime Nazar Anchorena

Contents

Unidad 1: Presentacion **2**
Entorno de desarrollo 2
Crear procesos 2
`ps` 3
Copy on write 3

Unidad 1: Presentacion

Entorno de desarrollo

Trucos de bash:

- `ctrl + r` para buscar historial
- `!!` expande con el ultimo comando (`$sudo !!`)
- `file.{jpg, png}` expande a `file.jpg file.png`
- `$(...)` da el output de un comando (`touch file-$(date -I)`)
- `ctrl + a` para ir al comienzo de la linea
- `ctrl + e` para ir al final de la linea
- `cd -` vuelve al dir que estabas antes
- `fc` fix command, abre el ultimo comando en un editor y corre la version editada
- `type` indica de si algo es builtin, program, o alias (`type ping`)

En bin/ estan todos los comandos de administracion, para configurar el sistema. Por otro lado, en bin/ estan los comandos de uso general.

Crear procesos

Se pueden crear utilizando `fork`, syscall que la llamo una vez y vuelve dos veces, el proceso original y el que se creo.

Los dos procesos son identicos salvo el valor de retorno de `fork`. Si `fork` no puede crear un proceso(raro, pero puede pasar), devuelve -1.

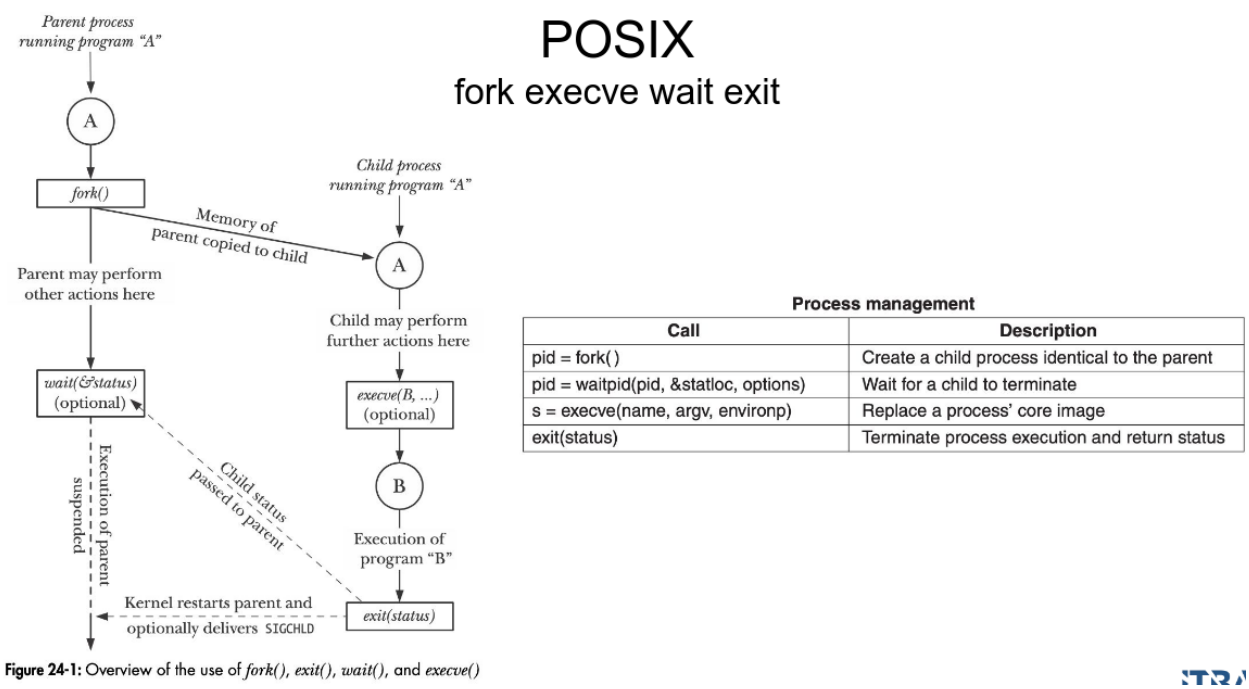

Figure 1: Pipeline fork

Como cada proceso puede crear varios procesos, pero tiene un solo padre, si quiero saber cual es el `pid` del

padre es necesario que el padre sepa cual es el `pid` del hijo. Esto se logra devolviendo el `pid` del hijo en el padre.

Es muy importante la llamada a `wait` porque sino queda un monton de procesos **undefined**, consume entradas en la tabla de procesos. Todas las variables que tenga un proceso original, incluyendo los punteros, el hijo los hereda. Es importante por ese motivo cerrar todos los **file descriptor**.

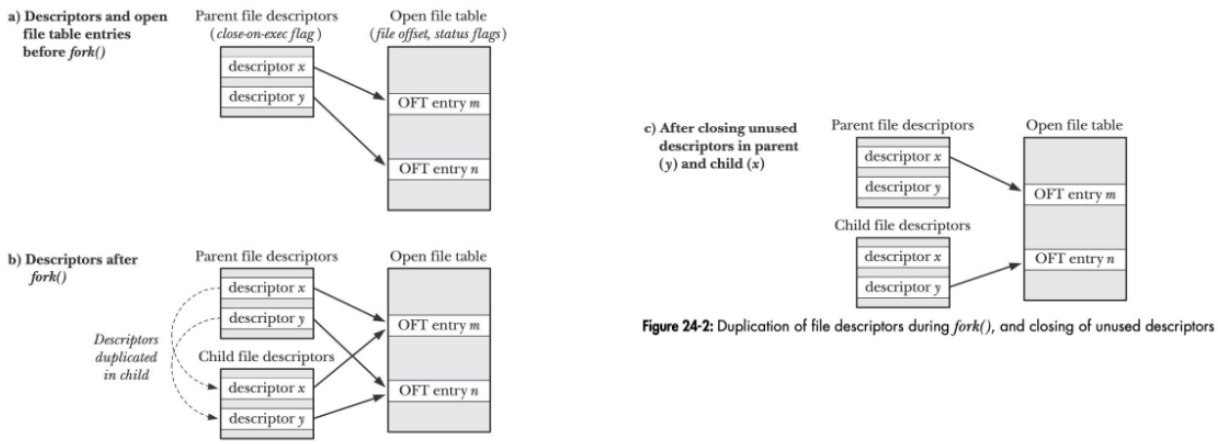

Figure 2: File descriptors fork

`ps`

Muy recomendado que lo aprendamos a usar bien para debuggear los TP. Se corre generalmente con `aux` pued indica: a+x -> mostrar todos los procesos, u -> incluir la columna con informacion del usuario de cada programa. Se puede combinar con otros arg(todo junto):

- Muestra los procesos que estan corriendo en el momento. Usando `w` como argumento(wide) muestra los argumentos que se utilizaron para correr cada programa.

- Si se corre con `e` va a mostrar las variables de entorno.
- Si se corre con `f` se mostrara un arbol de los procesos con caracteres ASCII

  Nota: Hay mas arg en la presentacion de campus.

Copy on write

Los procesos clones usan la misma memoria que el padre(heap, stack, memory maps), copiar esa memoria cada vez que usamos fork es lento y ocupa memoria. Entonces, linux permite que ambos procesos utilicen la misma memoria fisica y solo copia la memoria cuando alguno de los procesos intenta escribir. Linux implementa esta dandole la misma **page table** a cada programa, pero marcadas como **read only**. Entonces, cuando alguno intenta escribir ocurre una **page fault**.

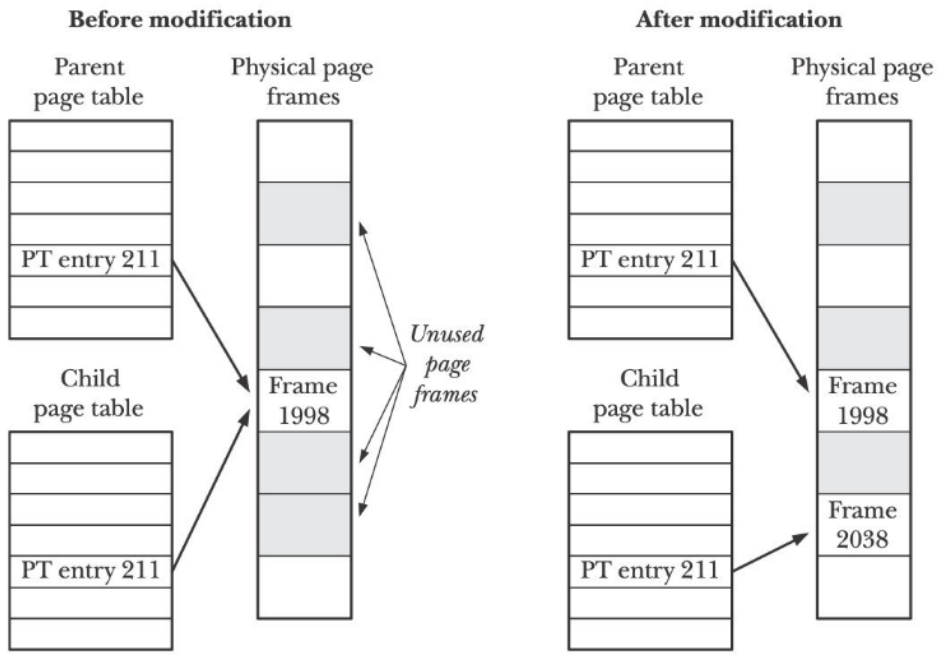

Figure 24-3: Page tables before and after modification of a shared copy-on-write page

Figure 3: Representacion de copy on write